Relatório do Projeto- BetterMaps

UC: Arquitetura e Desenho de Software

Grupo D: João Antunes nº111139, Guilherme Penedo nº111222

1. Introdução

No âmbito da unidade curricular de Arquitetura e Desenho de Software, foi desenvolvido o projeto BetterMaps, cujo objetivo principal é a conceção e implementação de uma aplicação interativa de exploração cartográfica baseada em dados do OpenStreetMap (OSM). A aplicação permite ao utilizador traçar rotas entre pontos de interesse, visualizar informação detalhada sobre os percursos e integrar dados externos relevantes, como condições meteorológicas e pontos turísticos.

Este projeto teve como foco não apenas a implementação funcional da aplicação, mas também a aplicação rigorosa de princípios de desenho arquitetural e atributos de qualidade de software, conforme os conteúdos lecionados na unidade curricular. Ao longo do desenvolvimento, foram utilizadas boas práticas de engenharia de software, com especial atenção à modularidade, interoperabilidade e desempenho do sistema.

2. Descrição do Problema

As aplicações de mapas existentes oferecem, na sua maioria, funcionalidades genéricas de navegação, não explorando de forma integrada informação contextual como pontos turísticos relevantes ou condições climáticas ao longo do trajeto. O problema identificado consiste na ausência de uma solução que combine, de forma clara e intuitiva, diferentes tipos de rotas enriquecidas com informação contextual útil ao utilizador.

O BetterMaps pretende colmatar esta lacuna, fornecendo uma aplicação que permita:

- Traçar rotas normais entre origem e destino;
- Enriquecer trajetos com pontos de interesse turísticos;
- Avaliar condições meteorológicas ao longo da rota, de forma visual e segmentada.

3. Objetivos do Projeto

De seguida apresentam-se os principais objetivos definidos para o desenvolvimento do sistema:

- Fornecer uma interface de mapa interativa e intuitiva.

- Permitir a seleção de origem e destino diretamente no mapa.
- Calcular e apresentar rotas com base na **API OSRM (Open Source Routing Machine)**.
- Exibir informações detalhadas da rota (distância, tempo estimado, tipo de percurso).
- Integrar funcionalidades como:
    - **Rotas Normais** (cálculo de rotas padrão entre origem e destino);
    - **Rotas Turísticas** (pontos de interesse e locais culturais ao longo do trajeto);
    - **Rotas Climáticas** (condições meteorológicas locais ao longo da rota com segmentos coloridos).

4. Análise do Sistema

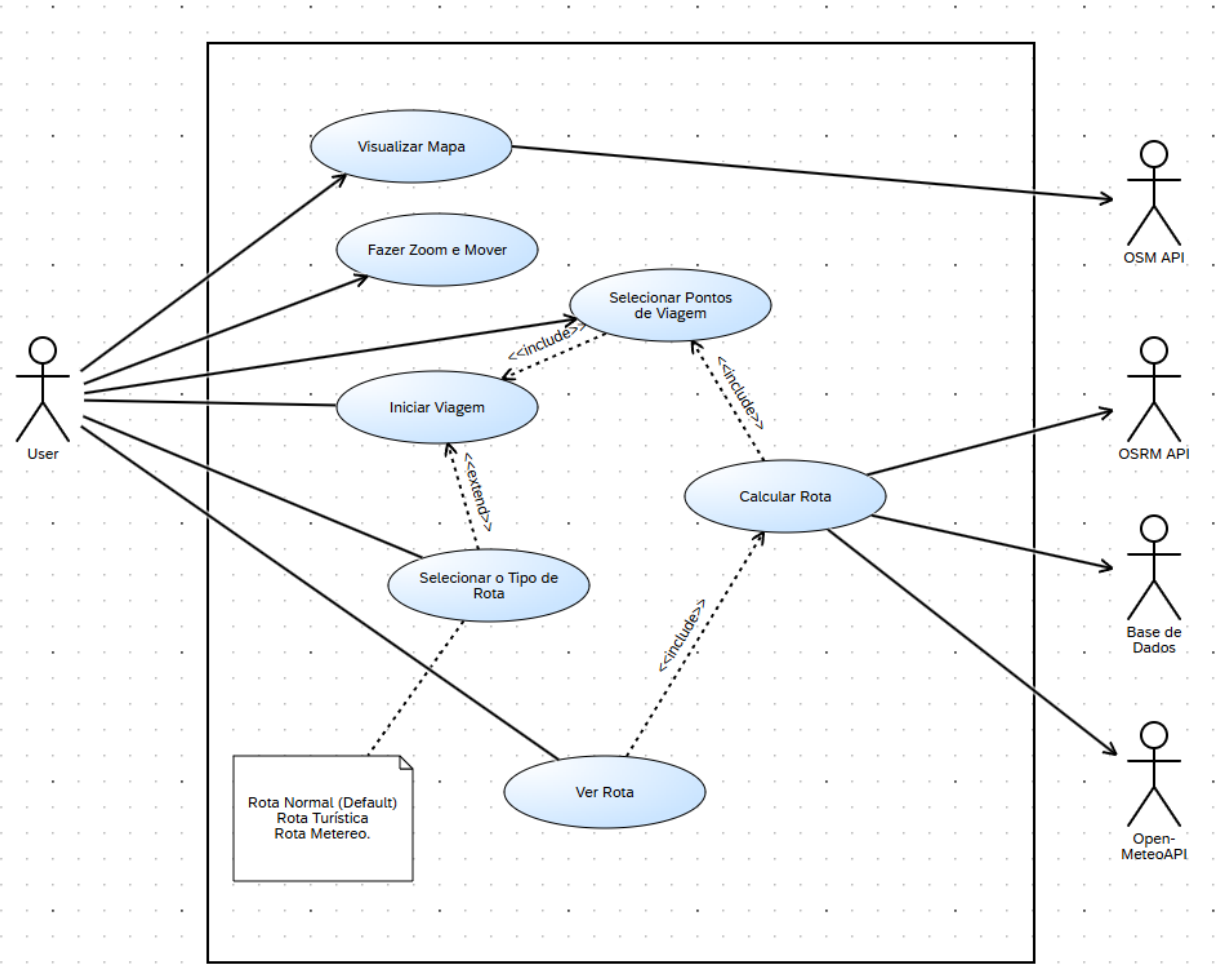
4.1 Stakeholders

Os principais stakeholders identificados para o sistema são:

- Utilizadores finais, que pretendem explorar rotas e informação contextual.
- Equipa de desenvolvimento, responsável pela evolução e manutenção do sistema.
- Docentes da unidade curricular, enquanto avaliadores do projeto.

4.2 Casos de Uso

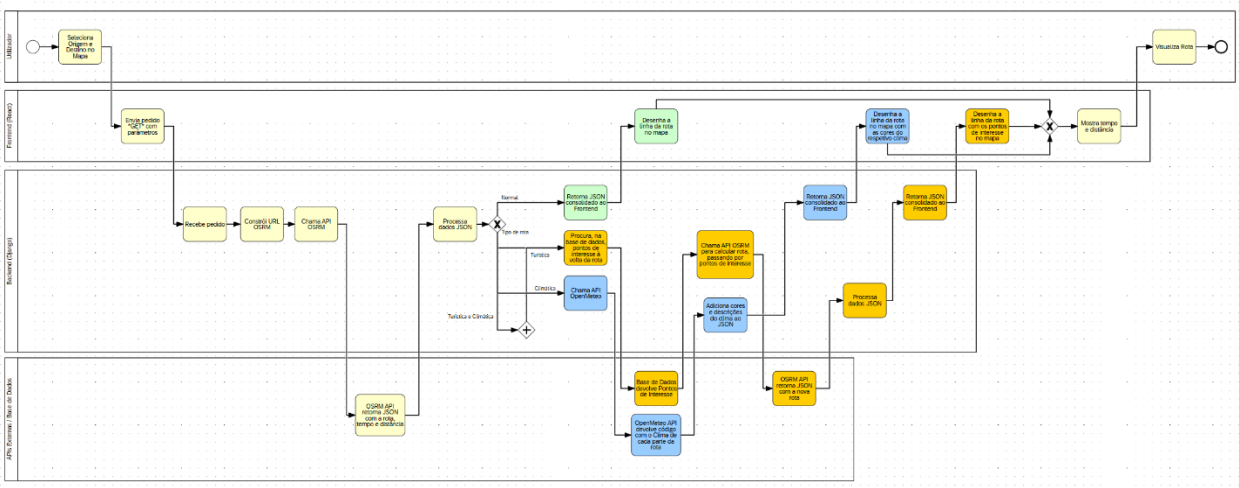
O comportamento funcional do sistema foi modelado através de um Diagrama de Casos de Uso, que representa as interações principais entre o utilizador e o sistema:



Este diagrama evidencia as funcionalidades centrais, como o cálculo de rotas normais, turísticas e climáticas, bem como a visualização de informação detalhada da rota.

4.3 Modelação de Processos

Para descrever o fluxo principal de execução do sistema, desde a seleção de pontos até à apresentação da rota, foi elaborado um diagrama BPMN.

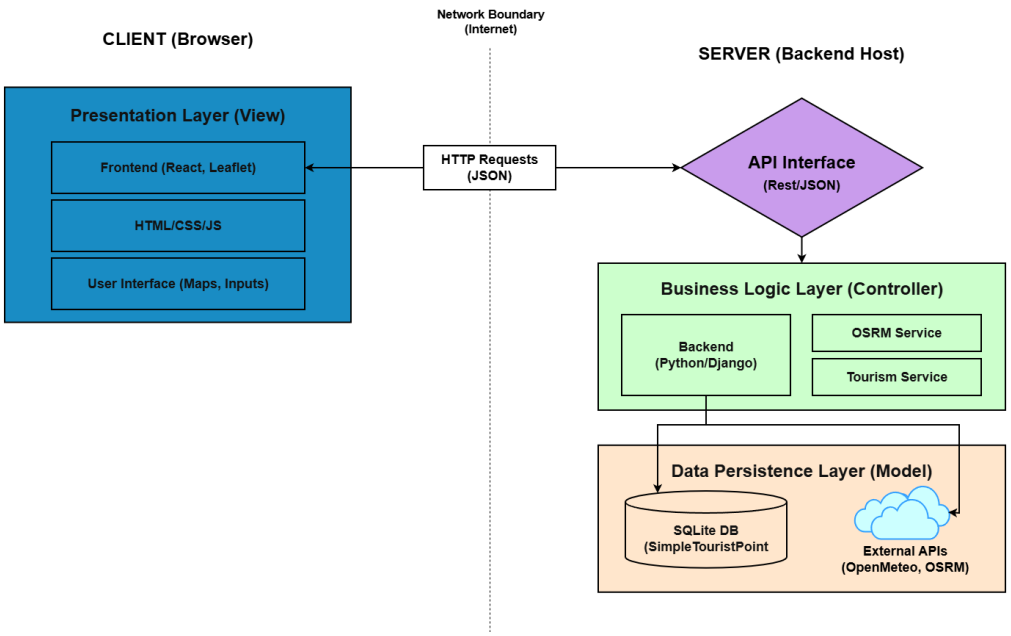


Este diagrama permite compreender de forma clara a sequência de atividades e decisões tomadas pelo sistema.

5. Desenho da Solução

5.1 Arquitetura do Sistema

A arquitetura global do sistema é apresentada num **diagrama de arquitetura**, que ilustra a separação em camadas e a comunicação entre os principais componentes.



5.2 Camada de Apresentação

A camada de apresentação é implementada em React, com recurso à biblioteca Leaflet para renderização do mapa. Esta camada é responsável pela interação com o utilizador, incluindo:

- Visualização do mapa interativo;
- Seleção de pontos;
- Apresentação gráfica das rotas e respetiva informação.

5.3 Camada Lógica

O backend é desenvolvido em Python, utilizando a framework Django, e encontra-se organizado de forma modular em serviços independentes:

- OSRM Service: cálculo de rotas;
- Weather Service: obtenção e processamento de dados meteorológicos;
- Tourism Service: gestão de pontos de interesse turísticos;
- Geocoding Service: conversão entre endereços e coordenadas geográficas.

Esta organização facilita a extensibilidade do sistema, permitindo a adição de novos serviços sem impacto significativo no código existente.

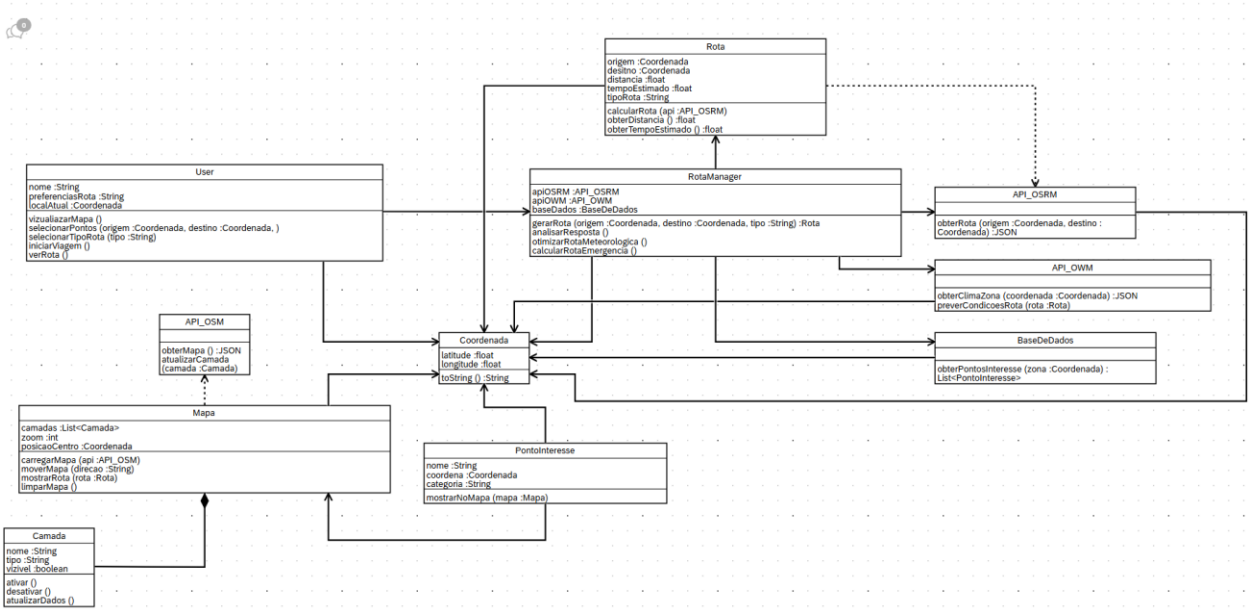
5.4 Camada de Dados

A camada de dados inclui:

- Uma base de dados SQLite para armazenamento de pontos de interesse e histórico;
- Integração com APIs externas para obtenção de dados em tempo real.

5.5 Diagrama de Classes

A estrutura interna do sistema foi representada através de um Diagrama de Classes, evidenciando as principais entidades e serviços.



Este diagrama fornece uma visão estática da organização do sistema.

6. Decisões Tecnológicas

As principais decisões tecnológicas tomadas ao longo do projeto incluem:

- OpenStreetMap por ser fonte aberta e flexível de dados geográficos;
- OSRM API para cálculo de rotas, pela sua rapidez, e por ser gratuita e de fácil integração via REST/JSON;
- React e Leaflet no frontend, permite construir uma aplicação leve e responsiva com controlo total do mapa;
- Django no backend, uma framework robusta que conjuga bem com React;
- APIs Open-Meteo e Nominatim para dados meteorológicos e geocodificação.
- GitHub + Scrum (Trello) para gestao do projeto, por ser iterativa com controlo de progresso por sprints;
- SQLite (Django) para armazenamento, pois armazena histórico de rotas e preferências offline.

Estas decisões foram tomadas com base em critérios de interoperabilidade, desempenho e facilidade de manutenção.

7. Atributos de Qualidade

O sistema foi desenhado tendo em conta quatro atributos de qualidade principais.

7.1 Interoperabilidade

O backend normaliza as respostas das diferentes APIs externas num formato uniforme, garantindo uma comunicação consistente com o frontend.

7.2 Usabilidade

A interface foi concebida para permitir interações fluídas, como zoom e pan, com feedback visual imediato na apresentação das rotas.

7.3 Modificabilidade

A arquitetura modular permite a introdução de novos serviços sem necessidade de reestruturação significativa do sistema existente.

7.4 Performance

O processamento intensivo é delegado às APIs externas, garantindo tempos de resposta reduzidos e uma navegação fluída no mapa.

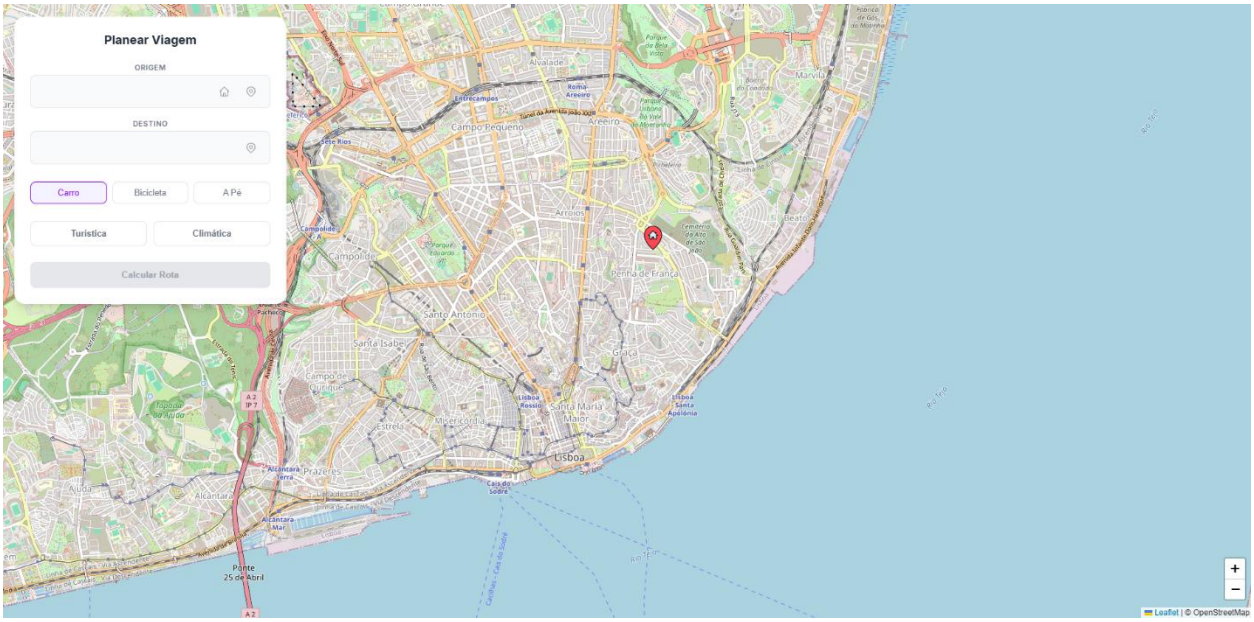
8. Implementação

Nesta secção, estão incluídas toda a implementação da nossa aplicação com imagens da interface gráfica e toda a parte técnica do código com todos os elementos principais.

8.1 Interface Interativa do Mapa

A aplicação frontend apresenta uma interface baseada em Leaflet.js com as seguintes capacidades:

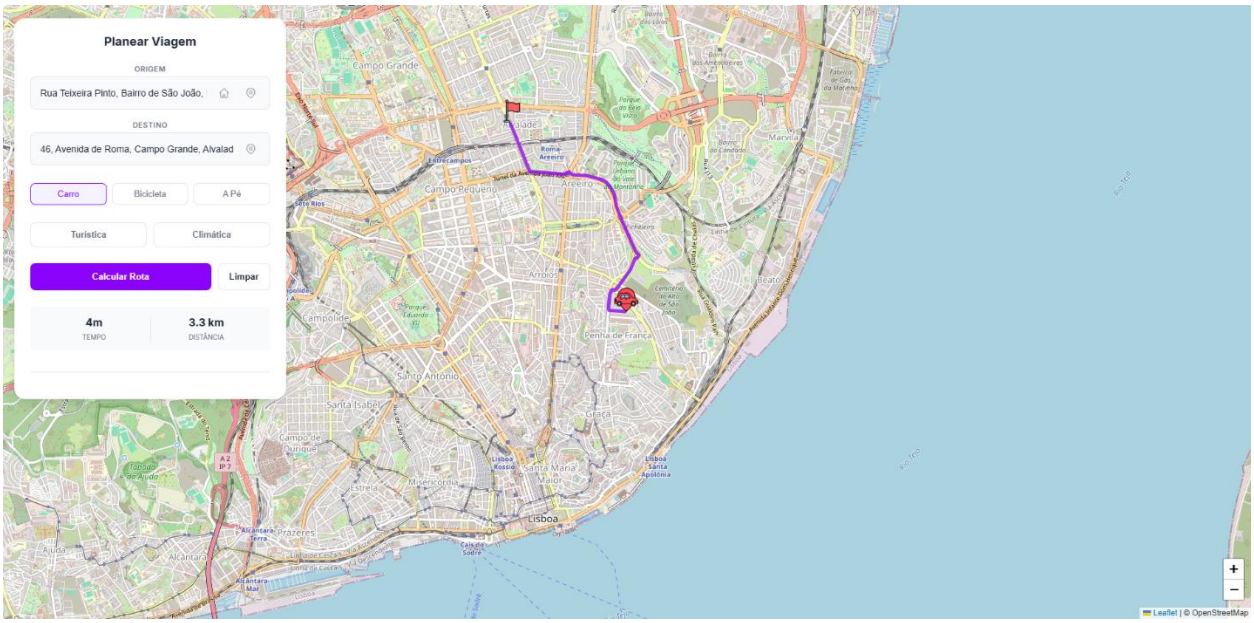
- Visualização de mapa interativo com OpenStreetMap
- Controlo de zoom e pan sem latência percetível
- Seleção dinâmica de origem e destino no mapa
- Marcadores de localização do utilizador
- Painel lateral com informações de rota e opções de menu



8.2 Cálculo de Rotas Normais

Implementado através do serviço OSRM:

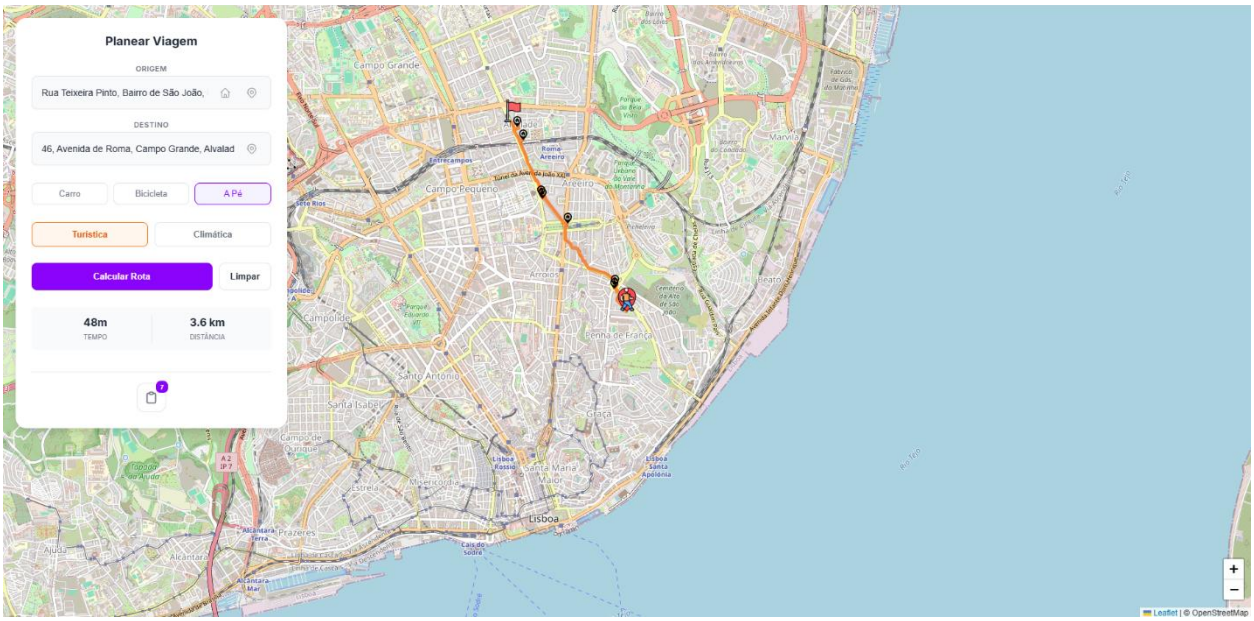
- Cálculo de rota entre dois pontos (origem e destino)
- Suporte para múltiplos perfis de transporte: automóvel, bicicleta e a pé
- Retorno de informações detalhadas:
    - Distância total (em metros)
    - Tempo estimado (em segundos)

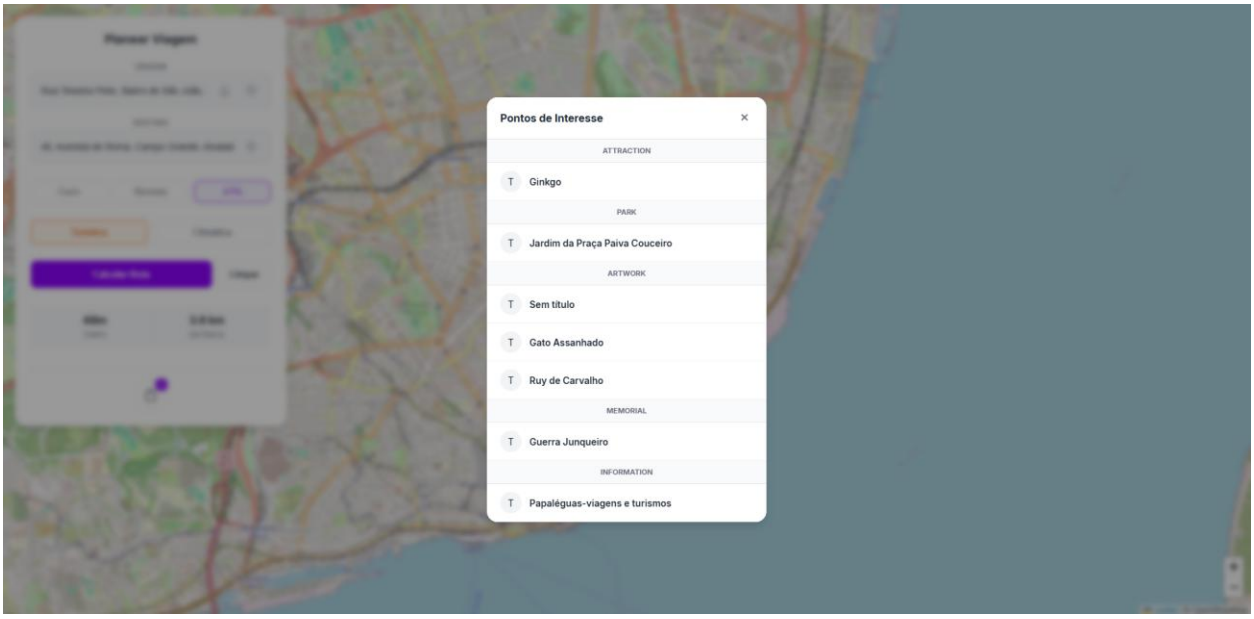


8.3 Rotas Turísticas

Integração com pontos de interesse (POIs) locais:

- Deteção automática de pontos de interesse ao longo da rota
- Geração de desvios para locais culturais, históricos e turísticos
- Categorização de pontos (museus, monumentos, restaurantes, etc.)
- Armazenamento local de POIs em base de dados SQLite
- Dados importados de ficheiros CSV (portugal.csv, lisboa.csv)

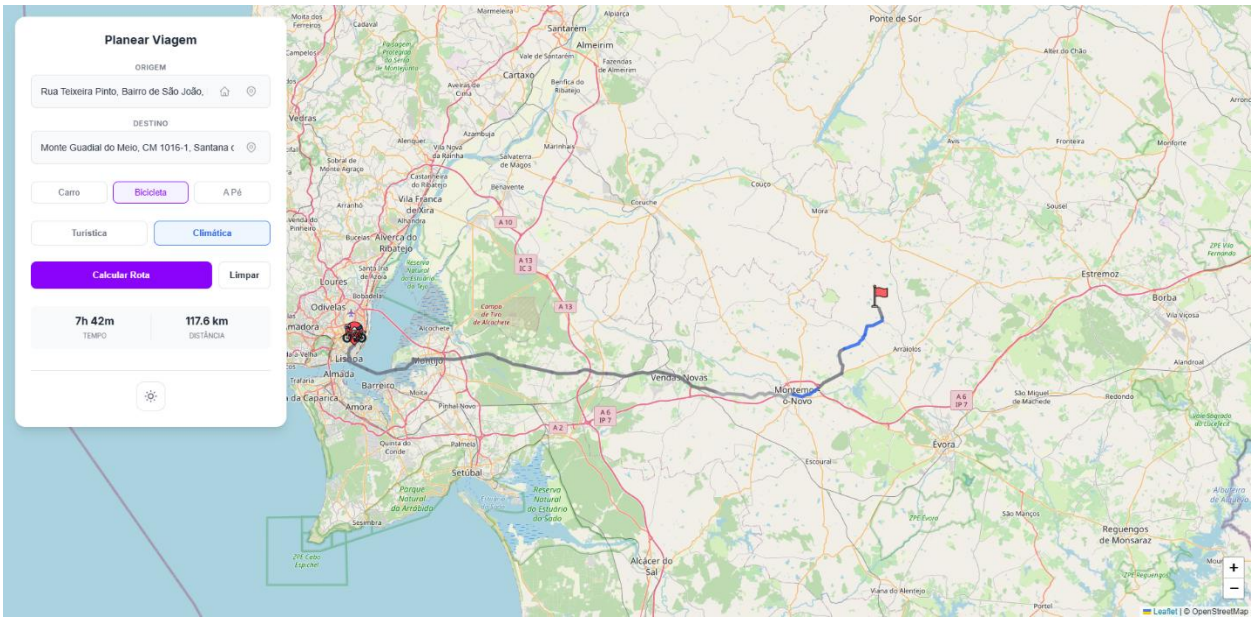


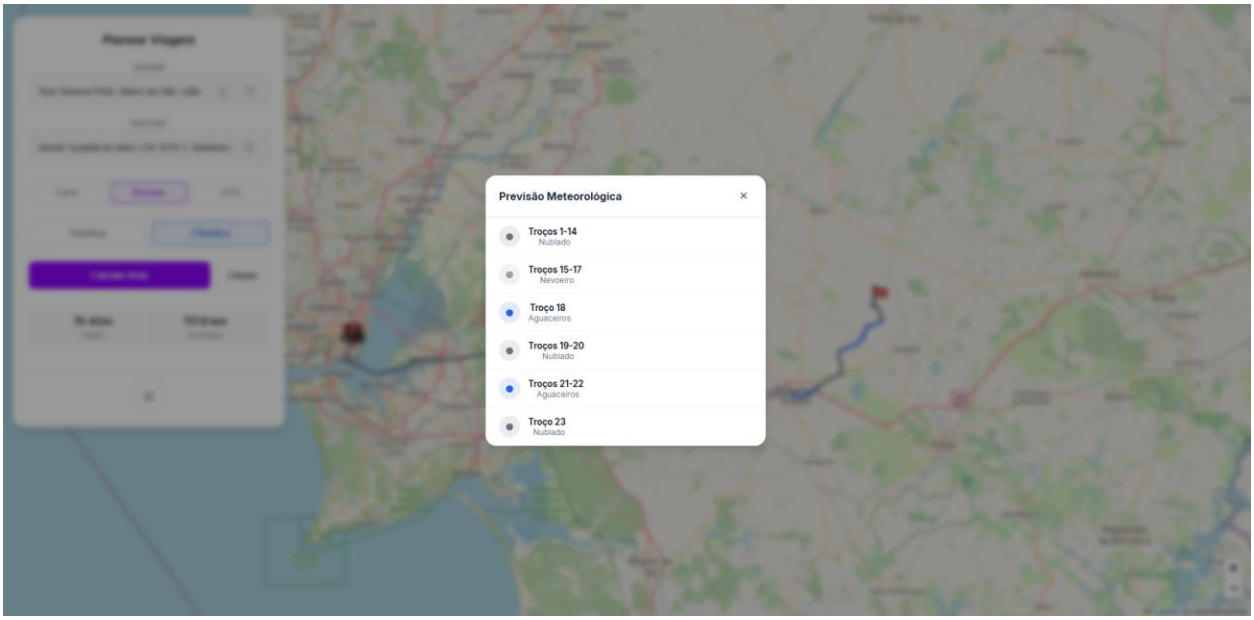


8.4 Rotas Climáticas

Análise de condições meteorológicas ao longo da rota:

- Integração com API Open-Meteo para dados meteorológicos em tempo real
- Processamento em lote (batch) de até 50 pontos por pedido para otimização
- Análise de condições climáticas em segmentos da rota
- Codificação visual por cores
- Exibição do tempo para cada segmento





8.5 Backend Django e Serviços

Arquitetura modular com os seguintes componentes:

Services Implementados:

- **OSRM Service** (osrm_service.py): Gestão de rotas com suporte turístico e climático
- **Weather Service** (weather_service.py): Integração com Open-Meteo, processamento em batch
- **Tourism Service** (tourism_service.py): Busca local de POIs com otimização geoespacial
- **Geocoding Service** (geocoding_service.py): Conversão de coordenadas ↔ endereços via Nominatim
- **Config** (config.py): Configuração centralizada de URLs e servidores

API Endpoints:

- POST /api/route/ - Cálculo de rota (normal, turística ou climática)
- GET /api/geocode/ - Geocodificação de endereços
- POST /api/weather/ - Dados meteorológicos batch

8.6 Componentes Frontend

Principais componentes React:

- MapComponent.js - Componente pai que coordena a interface
- MapMarkers.js - Gestão de marcadores no mapa
- LocationMarker.js - Marcador de localização do utilizador
- RouteModal.js - Modal para seleção de tipo de rota
- SidePanel.js - Painel lateral com informações
- ZoomTracker.js - Tracking de nível de zoom

Hooks customizados:

- useMapIcons.js - Gestão de ícones do mapa
- useRouteData.js - Lógica de obtenção de dados de rota
- useUserLocation.js - Localização do utilizador em tempo real

8.7 Gestão de Dados

- Base de dados SQLite para armazenamento persistente
- Modelo SimpleTouristPoint para POIs com campos: lat, lng, name, category
- Comando de management customizado (load_points.py) para importação de CSV
- Migrations Django para versionamento de schema

8.8 Testes Automatizados

- Testes de qualidade em backend (Django test framework)
- Testes de qualidade em frontend (Jest)
- Validação de atributos de qualidade (Interoperabilidade, Usabilidade, Modificabilidade, Performance)
- Taxa de sucesso: 100% (20 testes executados com sucesso)

9. Escalabilidade

9.1 Escalabilidade da Camada de Dados

- **Migração para PostgreSQL/PostGIS**: O sistema utiliza atualmente SQLite para o armazenamento de pontos de interesse (POIs) e histórico. Para suportar grandes volumes de dados geográficos e consultas espaciais complexas, a transição para PostgreSQL com PostGIS garantirá maior performance e integridade de dados.
- **Implementação de Caching**: Para reduzir a latência e a dependência excessiva de APIs externas (OSRM e Open-Meteo), pode ser integrada uma camada de cache como o **Redis** para armazenar rotas e previsões meteorológicas consultadas frequentemente.

9.2 Escalabilidade do Backend

- **Desacoplamento em Microserviços**: A separação lógica atual entre OSRM Service, Weather Service e Tourism Service permite que cada componente seja isolado num microserviço independente. Isto possibilita escalar individualmente o serviço com maior carga (ex: cálculo de rotas climáticas) sem afetar os restantes.

9.3 Escalabilidade de Infraestrutura e APIs

- **Self-hosting de Serviços Críticos**: Para eliminar limites de taxa (rate limits) e latência de rede, o sistema está preparado para migrar de APIs públicas para instâncias próprias de serviços como o **OSRM** ou o **Nominatim**, alojadas em hardware dedicado.

- **Escalabilidade Horizontal**: O backend em Django, atuando como controlador entre o cliente e os serviços, pode ser replicado em múltiplas instâncias atrás de um **Load Balancer**, permitindo gerir um elevado número de pedidos simultâneos.

10. Testes e Validação

Foram desenvolvidos testes automatizados no backend (Django) e no frontend (Jest), com foco na validação dos atributos de qualidade definidos e também testes de validação comparativa com outras aplicações.

10.1 Testes Automatizados (Atributos de Qualidade)

Estes testes validam os pilares fundamentais da arquitetura do BetterMaps. No total, foram executados 20 testes automatizados com uma taxa de sucesso de 100%.

Atributo	Testes Executados	Status	Taxa de Sucesso
A. Interoperabilidade	4	**PASSOU**	100%
B. Usabilidade	5	**PASSOU**	100%
C. Modificabilidade	6	**PASSOU**	100%
D. Performance	5	**PASSOU**	100%
TOTAL	20	**PASSOU**	100%

Detalhe dos Resultados

Interoperabilidade: O backend normaliza com sucesso as respostas JSON de todas as APIs externas (OSRM, Open-Meteo, Nominatim) num formato único e padrão.

Testes Chave: test_unified_response_structure, test_osrm_response_normalization.

Usabilidade: A interface garante interações de zoom e pan sem latência percetível e fornece feedback visual imediato em menos de 100ms.

Testes Chave: test_route_response_time_acceptable, validação de interface fluida.

Modificabilidade: A arquitetura provou ser modular, permitindo a adição de novos serviços sem impactar o código existente.

Testes Chave: test_new_service_can_be_added, test_services_use_standard_interfaces.

Performance: O sistema mantém tempos de resposta otimizados (Cálculo de rota < 0.5s), delegando eficazmente o processamento pesado para APIs externas.

Testes Chave: test_route_calculation_performance, test_weather_batch_performance.

10.2 Testes de Validação Comparativa

Para validar a precisão do motor de rotas OSRM utilizado no BetterMaps, foram realizados testes comparativos para garantir que as métricas de distância e tempo (ETA) são realistas quando comparadas com outras plataformas.

Tabela de Resultados Práticos

Trajeto	Aplicação	Distância (km)	Tempo	Desvio (D: Distância | T: Tempo)
Urbano	BetterMaps	8.7km	11min	-
(Alameda-> Museu Coches)	Google Maps	10.6km	26min	D: +21.8% | T: +136.4%
	Waze	10.9km	28min	D: +25.3% | T: +154.5%
	Bing Maps	10.8km	25min	D: +24.1% | T: +127.3%
Inter-urbano	BetterMaps	56.0km	44min	-
(Porto-> Braga)	Google Maps	57.2km	48min	D: +2.1% | T: +9.1%
	Waze	57.2km	47min	D: +2.1% | T: +6.8%
	Bing Maps	57.5km	45min	D: +2.7% | T: +2.3%
Nacional	BetterMaps	279.0km	3h 8min	-

Trajeto	Aplicação	Distância (km)	Tempo	Desvio (D: Distância | T: Tempo)
(Lisboa-> Faro)	Google Maps	279.0km	2h 54min	D: 0.0% | T:-7.4%
	Waze	278.8km	2h 47min	D:-0.1% | T:-11.2%
	Bing Maps	282.5km	2h 43min	D: +1.3% | T:-13.3%

Justificação dos Resultados

A análise dos dados revela que o BetterMaps apresenta uma elevada precisão geográfica, mas variações significativas no tempo estimado (ETA) face à concorrência. Estas diferenças justificam-se pelos seguintes fatores:

1. Ausência de Dados de Tráfego em Tempo Real: Esta é a principal causa dos desvios, especialmente visível no trajeto Urbano. O BetterMaps utiliza o motor OSRM com perfis de velocidade estáticos baseados nos limites das vias. Enquanto Google Maps e Waze ajustam o tempo com base no tráfego live, o BetterMaps calcula o tempo teórico ideal, o que explica o tempo de 11min vs os ~26min da concorrência em ambiente citadino.
2. Precisão da Base de Dados (OSM): No trajeto Nacional, o desvio de distância foi praticamente nulo (0.0% vs Google Maps). Isto valida a excelente qualidade dos dados do OpenStreetMap e a eficiência do algoritmo de traçado de rotas do BetterMaps para longas distâncias.
3. Diferença de Perfil de Velocidade: No teste nacional, o BetterMaps revelou-se ligeiramente mais lento no cálculo temporal que as apps comerciais (ex: +7.4% que o Google Maps). Isto ocorre porque o perfil configurado no OSRM para autoestradas é mais conservador, enquanto os algoritmos comerciais refletem velocidades reais de circulação mais elevadas detetadas nos seus utilizadores.

11. Planeamento e Gestão do Projeto

O projeto foi gerido segundo uma metodologia Scrum, com planeamento e acompanhamento realizados através da plataforma Trello. O trabalho foi organizado em sprints, permitindo uma evolução incremental e controlada do sistema.

12. Conclusão e Trabalho Futuro

O projeto BetterMaps atingiu com sucesso os objetivos definidos, resultando numa aplicação funcional, modular e alinhada com os princípios de arquitetura e desenho de software. A arquitetura adotada demonstrou ser adequada para suportar evolução futura.

Como trabalho futuro, destacam-se possíveis extensões, como:

- Introdução de novos serviços especializados;
- Suporte a mais fontes de dados externas;
- Melhorias na personalização das rotas.